Software Requirements
Specification

A Mobile Application for Facility Management

Team

[Secret]

Members
[Omar Khaled] | [13002892] (Team Leader)
[Farida Adham] | [14001734]
[Ziad Deifallah] | [13006806]
[Omar Ahmed Ismail] | [13006148]
[Marwan Ibrahim] | [14003376]

Tutorial Details
[T-01] | [Moamen Atia]

1. Introduction

Purpose

This document describes the Software Requirements Specification (SRS) for the CampusCare mobile application. It explains what the system should do and how it should work. It will be used by the development team, the TA, and the instructor during the project.

Overview of the Software

CampusCare is a mobile application that helps students and staff report maintenance problems on campus. For example, if someone sees a broken chair, water leak, or overflowing trash can, they can take a photo, write a short description, choose the location, and submit a ticket.

The Facility Management team can then view the ticket, assign a worker, and update the status until the issue is resolved. The goal of the system is to make reporting easier and improve the maintenance process on campus.

1.1 Product Vision & Scope

Product Vision

The vision of CampusCare is to create a simple and efficient system that connects the university community with the Facility Management team. The system aims to make maintenance faster, more organized, and more transparent.

Scope

The project includes:

- A mobile application built using React Native
- A backend system built using Node.js and REST APIs
- A PostgreSQL database to store data
- Cloud storage for uploaded images

The system focuses only on maintenance and facility issues inside the university campus.

1.2 Definitions and Acronyms

SRS: Software Requirements Specification — a document that describes the functional and non-functional requirements of a software system.

Ticket: A digital record created by a community member to report a facility issue. Each ticket contains a photo, description, location, category, and a status that is updated throughout its lifecycle.

RBAC: Role-Based Access Control — a security model in which system permissions are granted based on a user's assigned role (Community Member, Worker, Facility Manager, or System Admin).

API: Application Programming Interface — a set of protocols that allows the mobile application to communicate with the backend server.

RESTful API: A type of API that follows the Representational State Transfer architectural style, used in CampusCare for all client-server communication.

MVP: Minimum Viable Product — the core set of features required for the initial release of CampusCare, covering issue submission, assignment, status tracking, and notifications.

React Native: An open-source mobile application framework used to build the CampusCare mobile frontend for both iOS and Android platforms.

Expo: A toolchain built on top of React Native that simplifies mobile development, testing, and deployment.

Node.js: A JavaScript runtime environment used to build the CampusCare backend server.

PostgreSQL: An open-source relational database management system used to store all CampusCare data including users, tickets, locations, and notifications.

ERD: Entity Relationship Diagram — a visual representation of the database structure, showing tables, their attributes, data types, primary keys, foreign keys, and the relationships between them.

UML: Unified Modeling Language — a standardised set of diagrams used to model the structure and behaviour of a software system. In this document, a UML Use-Case Diagram is used to illustrate the interactions between actors and system features.

Actor: Any user or external system that interacts with CampusCare. The actors in this system are: Community Member, Worker, Facility Manager, System Admin, and Notification System.

Status: The current state of a submitted ticket. Possible values are: Pending, In Progress, and Resolved.

Notification: An automated system message sent to a user when a relevant event occurs, such as a ticket being submitted, assigned, or resolved.

2. Functional Requirements

1. Community Member Functional Requirements

FR-CM-01: Account Creation
The system shall allow a Community Member to create an account by providing required information (e.g., name, email, and password).

FR-CM-02: User Authentication
The system shall allow a Community Member to log in using a valid email and password.

FR-CM-03: Issue Submission
The system shall allow a Community Member to submit an issue including the following details:

- Location
- Photo
- Category
- Description

FR-CM-04: Issue Description
The system shall allow a Community Member to add a textual description when submitting an issue.

FR-CM-05: View Submitted Issues
The system shall allow a Community Member to view a list of issues they have submitted.

FR-CM-06: Logout
The system shall allow a Community Member to log out of the application.

FR-CM-07: View Issue Status
The system shall display the current status of an issue to the Community Member.
Possible statuses include:

FR-CM-08: Receive Notifications The system shall send a notification to a Community Member when the status of their submitted issue is updated.

- Pending
- In Progress
- Resolved

---

2. Facility Manager Functional Requirements

FR-FM-01: Account Creation
The system shall allow a Facility Manager to create an account.

FR-FM-02: User Authentication
The system shall allow a Facility Manager to log in using a valid email and password.

FR-FM-03: Logout
The system shall allow a Facility Manager to log out of the application.

FR-FM-04: View All Issues
The system shall allow a Facility Manager to view all submitted issues in the system.

FR-FM-05: Update Issue Status
The system shall allow a Facility Manager to update the status of an issue.

FR-FM-06: Assign Issue to Worker
The system shall allow a Facility Manager to assign an issue to a specific worker.

FR-FM-07: Close Issue
The system shall allow a Facility Manager to close an issue once it has been resolved.

FR-FM-08: Set Issue Priority The system shall allow a Facility Manager to set the priority of an issue (Low, Medium, or High).

---

3. Worker Functional Requirements

FR-W-01: Account Creation
The system shall allow a Worker to create an account.

FR-W-02: User Authentication
The system shall allow a Worker to log in using a valid email and password.

FR-W-03: Logout
The system shall allow a Worker to log out of the application.

FR-W-04: View Assigned Issues
The system shall allow a Worker to view issues assigned to them.

FR-W-05: Comment on Issue
The system shall allow a Worker to add comments to an issue.

FR-W-06: Update Issue Status to In Progress
The system shall allow a Worker to mark an assigned issue as In Progress.

FR-W-07: Upload Completion Photo
The system shall allow a Worker to upload a photo after completing the work on an issue.

FR-W-08: Receive Assignment Notification The system shall send a notification to a Worker when a new issue is assigned to them.

---

4. System Admin Functional Requirements

FR-SA-01: Admin Authentication
The system shall allow a System Admin to log in.

FR-SA-02: Logout
The system shall allow a System Admin to log out.

FR-SA-03: Account Management
The system shall allow a System Admin to activate or deactivate user accounts.

FR-SA-04: View Registered Users
The system shall allow a System Admin to view all registered users in the system.

2.1 User stories:

Community Member

A GIU community member interacts with the system as the primary reporter of facility issues. For example, a student notices that a bathroom trash can is overflowing — she opens the application, submits a ticket by uploading a photo, selecting the building and room, choosing the appropriate category, and adding a short description. She immediately receives a notification confirming her submission. Over the next day, she checks back and sees the status update from Pending to In Progress, and finally receives a notification that the issue has been marked as Resolved. Another student, walking past a broken classroom projector, does the same — he logs in, submits the issue with its location and category, and tracks it from his submitted issues list until it is fixed.

Facility Manager

A GIU facility manager is responsible for overseeing all reported issues and ensuring they are handled efficiently. For example, a facility manager logs in each morning and reviews the list of newly submitted tickets. She notices a high-priority electrical issue in Building C and assigns it immediately to a qualified worker. Later that day, she spots a low-priority cleaning ticket and sets its priority accordingly before assigning it. When a worker comments that a job is done and uploads a completion photo, she reviews the evidence, updates the issue status, and closes the ticket — keeping the system accurate and up to date for everyone involved.

Worker

A GIU maintenance worker uses the system to receive and manage the tasks assigned to him. For example, a worker logs in and sees two newly assigned issues in his queue — a leaking pipe in the men's restroom and a broken door handle in the library. He opens the first ticket, marks it as In Progress so the facility manager and the reporting student are both aware work has begun, then heads to fix it. Once the repair is complete, he uploads a photo of the fixed pipe and leaves a comment confirming completion. He then moves to the next ticket and repeats the same process, keeping his task list clear and his progress visible.

System Administrator

A GIU system administrator manages the platform at the user level, ensuring that only authorised individuals have access to the system. For example, an administrator logs in and reviews the full list of registered users. He notices a former employee's worker account is still active and deactivates it immediately to prevent unauthorised access. On another occasion, a new facility manager joins the team — the administrator confirms the account is properly set up and active so she can begin using the system on her first day. The administrator does not report or manage issues directly but keeps the user base clean, secure, and properly maintained.

1. Community Member

US-CM-01: As a Community Member, I should be able to create an account so that I can log in to the application.

US-CM-02: As a Community Member, I should be able to log in using my email and password so that I can access the application.

US-CM-03: As a Community Member, I should be able to submit an issue with its details (location, photo, and category) so that the problem can be reported accurately.

US-CM-04: As a Community Member, I should be able to add a description to the issue so that the worker understands the problem clearly.

US-CM-05: As a Community Member, I should be able to view my submitted issues so that I can track their status.

US-CM-06: As a Community Member, I should be able to log out so that my data remains secure on shared devices.

US-CM-07: As a Community Member, I should be able to view the status of my issue (Pending, In Progress, Resolved) so that I know the progress of my request.

US-CM-08: As a Community Member, I should be able to receive notifications when the status of my submitted issue changes so that I am kept informed without having to check the application manually.

---

2. Facility Manager

US-FM-01: As a Facility Manager, I should be able to create an account so that I can log in to the application.

US-FM-02: As a Facility Manager, I should be able to log in using my email and password so that I can access the application.

US-FM-03: As a Facility Manager, I should be able to log out so that my data remains secure on shared devices.

US-FM-04: As a Facility Manager, I should be able to view all submitted issues so that I can assign them to workers.

US-FM-05: As a Facility Manager, I should be able to update an issue's status so that the system reflects the current state of the issue.

US-FM-06: As a Facility Manager, I should be able to assign an issue to a worker so that the worker can start fixing it.

US-FM-07: As a Facility Manager, I should be able to close an issue after it is resolved so that the system reflects that the issue is completed.

US-FM-08: As a Facility Manager, I should be able to set the priority of an issue (e.g., Low, Medium, High) so that workers can address the most critical problems first.

---

3. Worker

US-W-01: As a Worker, I should be able to create an account so that I can log in to the application.

US-W-02: As a Worker, I should be able to log in using my email and password so that I can access the application.

US-W-03: As a Worker, I should be able to log out so that my data remains secure on shared devices.

US-W-04: As a Worker, I should be able to view my assigned issues so that I can start working on them.

US-W-05: As a Worker, I should be able to comment on an issue to indicate that the work is completed so that the facility manager can update its status.

US-W-06: As a Worker, I should be able to mark an issue as "In Progress" so that the facility manager and community member know that it is being worked on.

US-W-07: As a Worker, I should be able to upload a photo after fixing the issue so that the facility manager can verify the work.

US-W-08: As a Worker, I should be able to receive a notification when a new issue is assigned to me so that I can begin working on it promptly.

---

4. System Administrator

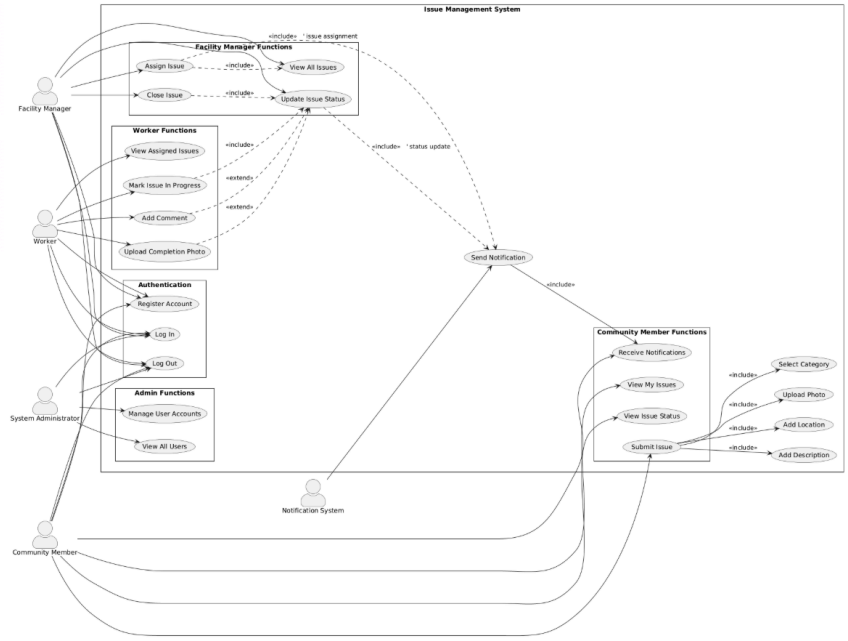
US-SA-01: As a System Admin, I should be able to log in so that I can access the application.

US-SA-02: As a System Admin, I should be able to log out so that my data remains secure on shared devices.

US-SA-03: As a System Admin, I should be able to activate or deactivate any account so that I can ensure all users follow the system rules.

US-SA-04: As a System Admin, I should be able to view all registered users so that I can manage the system users.

2.2 UML Use-case Diagram



3. Non-functional Requirements:

3.1 Performance

NFR-01 (Response Time): The system shall ensure that page load times are under three seconds under normal conditions.

NFR-02 (Concurrent Users): The system shall support at least 500 simultaneous users without significant performance degradation

3.2 Usability

NFR-03 (Ease of Use): The system shall provide an intuitive and user-friendly interface.
NFR-04 (Fast Submission): The system shall allow users to submit a maintenance ticket in under one minute.

3.3 Security

NFR-05 (Authentication): The system shall require user authentication before accessing system features.
NFR-06 (API Security): The system shall implement rate limiting and input validation to protect against misuse.
NFR-07 (Secure Communication): The system shall ensure secure data transmission using HTTPS.

3.4 Reliability

NFR-08 (System Availability): The system shall be available at least 99% of the time.
NFR-09 (Image Upload Stability): The system shall handle image uploads reliably, even under unstable network conditions.

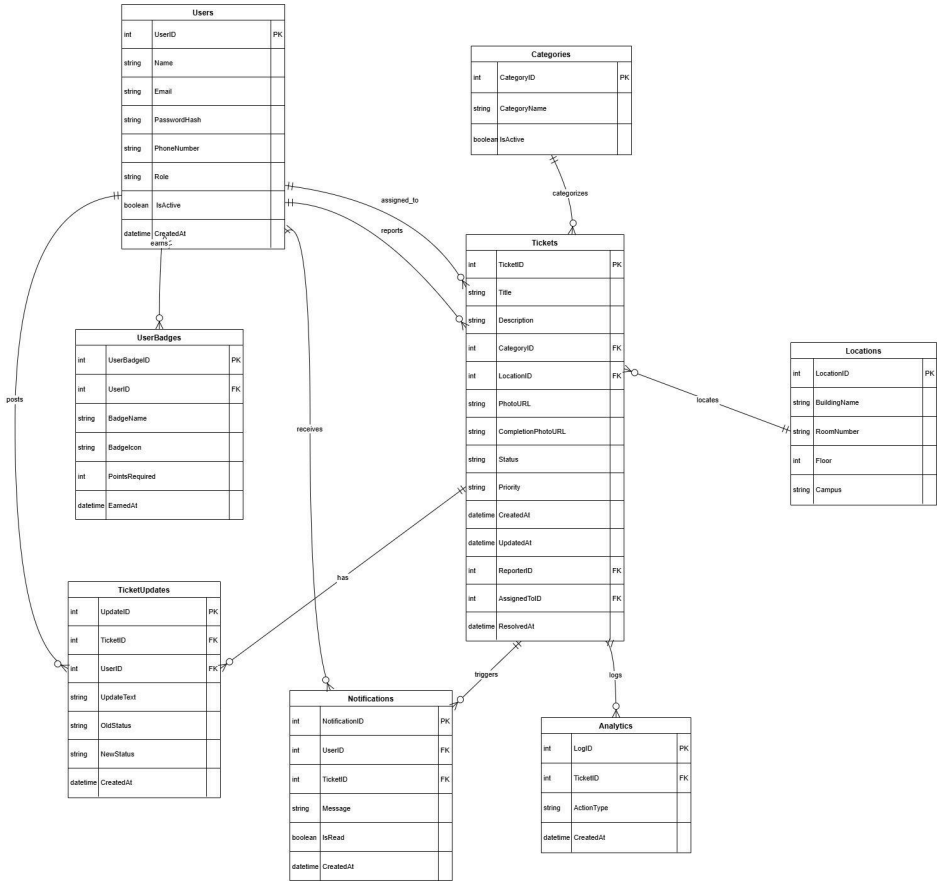
3.5 Backup & Recovery

NFR-10 (Data Backup): The system shall perform automated daily backups.
NFR-11 (Recovery Plan): The system shall include a disaster recovery plan to prevent data loss.

4. Constraints

This section describes the limitations and restrictions of the project.
C-01: The system shall be developed using React Native (Expo) for the mobile application.
C-02: The backend shall be developed using Node.js and RESTful APIs.
C-03: The system shall use PostgreSQL as the database.
C-04: The project must be completed within the academic semester timeline.
C-05: The system is limited to maintenance-related issues within the GIU campus only.

5. Entity Relation Diagram



6. Technology Stack

The system will be developed using the following technologies:

- Mobile Application: React Native (Expo)
- Backend: Node.js with RESTful APIs
- Database: PostgreSQL
- Image Storage: Cloud-based storage service
- Secure Communication: HTTPS

7. Future Scope of the Project

In future versions, the system may include:

- Push notifications for real-time updates
- Analytics dashboard for maintenance statistics
- Gamification features to encourage reporting
- AI-based prediction of frequently occurring issues
- Integration with university internal systems